Univerzitet u Zenici
Politehnički fakultet

BILJEŠKE S PREDAVANJA · SEDMICA 2

Uvod u IaC: virtualne mašine kao kod

Principi Infrastructure as Code pristupa (deklarativnost, idempotentnost, state, drift) i prvi praktični alati: `Terraform/OpenTofu`, `cloud-init` i `Packer`.

PREDMET	Računarstvo u oblaku
STUDIJ	II ciklus studija, III semestar, akademska godina 2026/27
NASTAVNIK	V. prof. dr. Đulaga Hadžić
ASISTENT	ass. Mr. Adin Jahić
TERMIN	petak, 23.10.2026, 17:15–20:15, Kabinet 3204
TEMA	Uvod u IaC: virtualne mašine kao kod

OKVIR NASTAVNE JEDINICE

Ciljevi i način rada

U prvoj sedmici je kontejner razložen na mehanizme jezgra, a hipervizor (KVM/QEMU, libvirt) predstavljen kao drugi način izolacije. Ova sedmica odgovara na praktično pitanje: *kako se virtualne mašine prave na način koji je ponovljiv, provjerljiv i verzioniran?* Odgovor je da se infrastruktura opisuje tekstualnim datotekama koje žive u Git repozitoriju, a alat zadužen za izvršavanje dovodi stvarno stanje u skladu s opisom.

Gradivo se izlaže od jednostavnog prema složenom. Prvi primjer ne zahtijeva ni oblak ni virtualizaciju (samo jedna datoteka) i služi da se na najmanjem mogućem slučaju vide svi principi. Svaki naredni primjer dodaje jedan sloj: cloud-init, pravu virtualnu mašinu i na kraju vlastitu sliku.

Nakon nastavne jedinice student je u stanju:

- objasniti razliku između imperativnog i deklarativnog pristupa i dati primjer za oba,
- definisati idempotentnost, state i drift te pokazati svaki od njih na radnom primjeru,
- napisati Terraform/OpenTofu konfiguraciju i voditi tok `init → plan → apply → destroy`,
- opisati šta radi cloud-init i napisati `user-data` koji korisnika, pakete i datoteke postavlja pri prvom pokretanju,
- kreirati virtualnu mašinu u libvirt-u iz koda i izgraditi vlastitu "zlatnu" sliku pomoću Packera,
- razlikovati ulogu Terraforma, Packera, cloud-inita i Ansible-a u istom lancu.

Preduslovi za izvođenje isječaka

- Linux ili WSL2 (Ubuntu 24.04) s pravom `sudo`. Za libvirt je u WSL2 potreban uključen `systemd` (`/etc/wsl.conf`: `[boot] systemd=true`) i `/dev/kvm` (ugniježđena virtualizacija).
- Terraform *ili* OpenTofu. Sve naredbe iz ovih bilješki rade i s `tofu` umjesto `terraform`.

- Primjeri 1 i 2 ne traže virtualizaciju. Primjeri 3 i 4 traže KVM; bez njega vidjeti napomenu uz primjer 3.
- Svaki primjer ima vlastiti direktorij pod `~/iac/s2/`.

bash — Isječak 0 · priprema

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients virtinst genisoimage curl gnupg
sudo usermod -aG libvirt,kvm $USER          # nakon toga odjava i ponovna prijava (ili: newgrp libvirt)

# HashiCorp repozitorij: Terraform i Packer
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform packer

ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519     # ključ koji će ući u virtualne mašine
terraform version && packer version
virsh -c qemu:///system list --all                   # provjera da libvirt radi
mkdir -p ~/iac/s2 && cd ~/iac/s2
```

---

1 · MOTIVACIJA

Zašto infrastruktura kao kod

Klasičan način rada je ručan: administrator se prijavi na server, instalira pakete, uredi konfiguraciju, otvori port. Taj pristup (u žargonu *ClickOps*) ima ponavljajuće probleme:

PROBLEM	POSLJEDICA
Ručne izmjene se ne bilježe	Nema odgovora na pitanje ko je, kada i zašto promijenio konfiguraciju.
Server je "snowflake" (jedinstven)	Ako ga izgubimo, ne znamo ga ponovo napraviti. Test i produkcija se razilaze.
Ponavljanje je sporo i podložno grešci	Deset servera znači deset ručnih postupaka i deset blago različitih rezultata.
Nema pregleda prije izmjene	Greška se otkriva tek kad nešto prestane raditi.

IaC to rješava tako što se željeno stanje infrastrukture zapisuje kao kod, a kod se tretira kao i svaki drugi softver: verzionira se u Gitu, pregleda (pull request), testira i izvršava automatski. Infrastruktura postaje **ponovljiva** (isti kod daje isti rezultat), **provjerljiva** (historija je u Gitu) i **dokumentovana** (kod je dokumentacija koja ne može zastarjeti).

> RADNA TEZA
>
> Infrastruktura se ne *održava*, ona se *opisuje*. Ako server treba promijeniti, mijenja se opis i ponovo se izvršava, a ne uređuje se server.

S tim u vezi stoji poznata metafora. Ako je server *kućni ljubimac* (pet), ima ime, njeguje se ručno i žali se kad pogine. Ako je *stoka* (cattle), ima broj, napravljen je iz koda i zamijeni se novim kad zakaže. IaC vodi prema stoci.

---

2 · PRINCIPI

Deklarativno, idempotentno, state, drift

2.1 Imperativno naspram deklarativnog

Imperativni pristup opisuje *korake* ("napravi ovo, pa ovo"). **Deklarativni** pristup opisuje *željeno stanje* ("treba postojati ovo"), a alat sam izračuna koji su koraci potrebni da se do njega dođe iz trenutnog stanja.

bash — Isječak 1 · imperativno (ilustracija)

```bash
#!/bin/bash
# kreiraj_vm.sh: drugo pokretanje puca jer VM već postoji
virt-install --name web1 --memory 1024 --vcpus 1 \
  --disk size=10 --import --os-variant ubuntu24.04 --noautoconsole
```

hcl — Isječak 2 · deklarativno (ilustracija)

```hcl
resource "libvirt_domain" "web1" {
  name   = "web1"
  memory = 1024
  vcpu   = 1
}
# "treba postojati VM web1 s 1 GiB memorije". Postoji li već, ne radi se ništa.
```

Skripta iz isječka 1 vrijedi samo za jedan početni slučaj: prvi put napravi VM, drugi put javlja grešku, a ako se memorija promijeni, ne pomaže ni to. Deklarativni zapis iz isječka 2 pokriva sva tri slučaja jer alat poredi željeno i stvarno stanje.

2.2 Idempotentnost

Operacija je **idempotentna** ako njeno ponovljeno izvršavanje daje isti rezultat kao jedno izvršavanje. U IaC-u to znači da se kod može bezbrižno pokrenuti više puta: ako je sistem već u željenom stanju, ništa se ne mijenja. To je preduslov za automatizaciju jer se isti kod izvršava pri svakoj izmjeni, a ne samo prvi put. Provjera je jednostavna: *drugo izvršavanje ne smije prijaviti nijednu promjenu*.

2.3 State

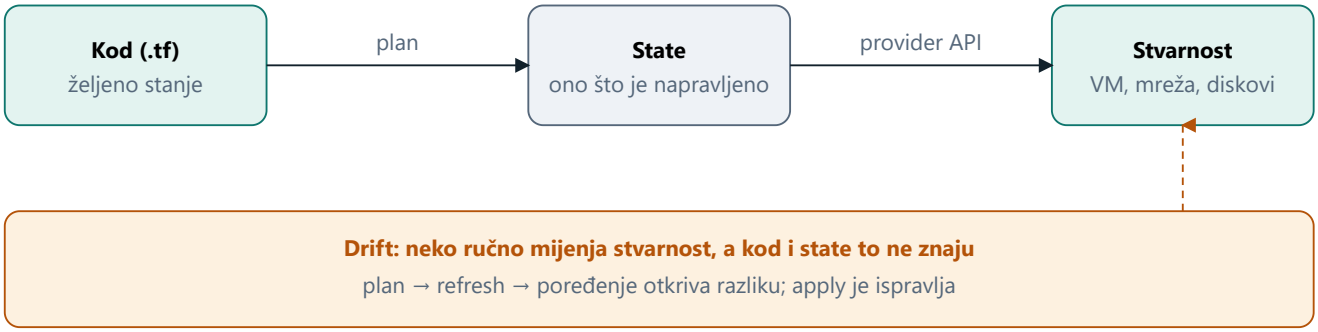
Alat mora znati šta je *on* napravio. Terraform to bilježi u datoteci **state** (`terraform.tfstate`) koja svaki resurs iz koda povezuje s konkretnim objektom u stvarnom svijetu (ID, atributi). Bez statea bi svaki `apply` morao pogađati šta je već napravljeno.

- State sadrži **osjetljive podatke** (lozinke, ključevi u čistom tekstu). Zato se nikad ne ubacuje u Git, već se dodaje u `.gitignore`.
- Pri radu u timu state se čuva u **udaljenom backendu** (npr. S3 ili HTTP backend) s *zaključavanjem*, da dvije osobe ne izvršavaju `apply` istovremeno.
- Pregled: `terraform state list`, `terraform state show <resurs>`. Postojeći objekt se preuzima u state naredbom `terraform import`.

2.4 Drift

Drift je razlika između stanja zapisanog u kodu i stvarnog stanja, nastala izmjenom izvan alata (neko ručno promijeni konfiguraciju ili obriše resurs). Terraform drift otkriva pri svakom `plan` pozivu tako što prvo osvježi state (*refresh*), a zatim ga poredi s kodom. Za samo otkrivanje, bez planiranja izmjena, služi `terraform plan -refresh-`

`only`. Ispravlja se tako što `apply` vraća stvarno stanje na ono iz koda (ili se kod prilagodi ako je promjena bila namjerna).



Slika 1. Tri izvora istine: kod, state i stvarnost. Terraform stalno poredi sva tri, a drift nastaje kad se promijeni stvarnost mimo koda.

2.5 Promjenjiva i nepromjenjiva infrastruktura

PRISTUP	IDEJA	ALAT
Promjenjiva (*mutable*)	Server postoji i na njemu se mijenjaju paketi i konfiguracija. Dugim radom se razilazi od opisa.	Ansible (sedmica 3)
Nepromjenjiva (*immutable*)	Server se ne mijenja. Za svaku izmjenu pravi se nova slika i zamijeni se stari server.	Packer + Terraform

U praksi se oba pristupa kombinuju: slika sadrži ono što se rijetko mijenja, a ono što zavisi od okruženja (hostname, ključevi, adrese) dodaje se pri prvom pokretanju, što je posao cloud-inita.

Alat i tok rada

Terraform (HashiCorp) je deklarativni alat koji infrastrukturu opisuje u jeziku **HCL** (*HashiCorp Configuration Language*). Nakon što je HashiCorp 2023. godine licencu promijenio u BSL, zajednica je pod okriljem Linux Foundation razvila **OpenTofu**, otvoreni fork. Za naše potrebe su sintaksa, tok rada i provideri isti; razlika je u binarnoj datoteci (`tofu`) i, od novijih mogućnosti, npr. ugrađenoj enkripciji statea u OpenTofu. Konfiguracije su između njih prenosive.

Građevni blokovi HCL-a

BLOK	ULOGA
`terraform { required_providers }`	Koji provideri i koje verzije se koriste. Provider je dodatak koji zna razgovarati s određenim sistemom (libvirt, Docker, AWS, DNS...).
`provider "x" { }`	Postavke providera (npr. adresa libvirt servisa).
`resource "tip" "ime" { }`	Objekt kojim se upravlja. Referencira se kao `tip.ime.atribut`.
`data "tip" "ime" { }`	Samo čitanje postojećeg objekta, bez upravljanja njime.
`variable "ime" { }`	Ulazni parametar (vrijednost iz `-var`, `terraform.tfvars` ili okruženja).
`output "ime" { }`	Vrijednost koja se ispisuje nakon `apply` (npr. IP adresa VM-a).
`locals { }`	Izračunate pomoćne vrijednosti unutar konfiguracije.

Redoslijed izvršavanja se *ne* piše ručno. Terraform iz referenci među resursima (npr. disk koji koristi bazni volumen) gradi graf zavisnosti, pa sam odredi poredak i paralelizam.

Tok rada

NAREDBA	ŠTA RADI
`terraform init`	Preuzima providere u `.terraform/` i zaključava verzije u `.terraform.lock.hcl` (ova datoteka ide u Git).
`terraform fmt` / `validate`	Uređuje format i provjerava sintaksu i reference.
`terraform plan`	Osvježava state, poredi s kodom i ispisuje *šta bi se uradilo*. Ništa ne mijenja. Oznake: `+` kreiranje, `~` izmjena, `-` brisanje, `-/+` zamjena.
`terraform apply`	Izvršava plan i upisuje rezultat u state.
`terraform destroy`	Uklanja sve resurse iz statea.

> **NAVIKA**
>
> `plan` se čita kao *pull request* za infrastrukturu. Prije svakog `apply` provjeriti da li plan radi samo ono što se očekuje, posebno je li negdje `-/+` ili `-` (brisanje podataka).

4 · PRIMJER 1

Terraform bez oblaka: idempotentnost, state i drift

Prvi primjer upravlja jednom tekstualnom datotekom pomoću providera `local`. Resurs je trivijalan, ali se na njemu vide sva četiri principa iz 2. poglavlja i ne treba ni virtualizacija ni račun u oblaku.

bash + hcl — Isječak 3 · prva konfiguracija

```
mkdir -p ~/iac/s2/p1 && cd ~/iac/s2/p1
cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

variable "ime" {
  type    = string
  default = "RuO"
}

resource "local_file" "pozdrav" {
  filename        = "${path.module}/pozdrav.txt"
  content         = "Zdravo, ${var.ime}!\n"
  file_permission = "0644"
}

output "putanja" {
  value = local_file.pozdrav.filename
}
EOF
terraform init
terraform plan
```

izlaz (skraćeno)

```
  # local_file.pozdrav will be created
  + resource "local_file" "pozdrav" {
      + content         = "Zdravo, RuO!\n"
      + filename        = "./pozdrav.txt"
      + file_permission = "0644"
      ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Idempotentnost

bash — Isječak 4 · dva puta apply

```
terraform apply -auto-approve     # 1. put: kreira datoteku
terraform apply -auto-approve     # 2. put: ništa za uraditi
cat pozdrav.txt
terraform state list
```

izlaz (skraćeno)

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.     # 1. izvršavanje
...
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.     # 2. izvršavanje
Zdravo, RuO!
local_file.pozdrav
```

Drugo izvršavanje prijavljuje `0 added, 0 changed`: kod je isti, stanje je isto, ništa se ne radi. To je idempotentnost. Naredba `state list` potvrđuje da je datoteka upisana kao resurs `local_file.pozdrav`.

Izmjena koda

bash — Isječak 5 · promjena ulazne varijable

```
terraform plan -var ime=Studenti
```

izlaz (skraćeno)

```
  # local_file.pozdrav must be replaced
-/+ resource "local_file" "pozdrav" {
      ~ content = <<-EOT # forces replacement
          - Zdravo, RuO!
          + Zdravo, Studenti!
        EOT
      ...
    }

Plan: 1 to add, 0 to change, 1 to destroy.
```

Plan odmah prikazuje *kakva* je promjena (`-/+`, zamjena resursa jer se sadržaj ovog providera ne može izmijeniti u mjestu), prije nego išta uradi. Kod pravih resursa će se često vidjeti i `~`, izmjena na licu mjesta.

Drift

bash — Isječak 6 · ručna izmjena izvan alata

```
echo "ručno izmijenjeno" > pozdrav.txt      # neko mijenja stvarnost mimo koda
terraform plan -refresh-only                # samo detekcija drifta
terraform apply -auto-approve               # vraća stanje na ono iz koda
cat pozdrav.txt
```

izlaz (skraćeno)

```
Note: Objects have changed outside of Terraform
  # local_file.pozdrav has been deleted
  - resource "local_file" "pozdrav" { ... }
...
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Zdravo, RuO!
```

DETALJ OVOG PROVIDERA

Provider `local` identitet resursa veže za hash sadržaja, pa izmijenjenu datoteku prijavljuje kao "obrisanu" i ponovo je kreira. Kod drugih providera (npr. memorija VM-a, pravilo vatrozida) isti drift se prikazuje kao `~ update in-place`. Princip je isti: razlika se uoči u `plan`, a `apply` je ispravlja.

bash — Isječak 7 · čišćenje i .gitignore

```
terraform destroy -auto-approve
cat > .gitignore <<'EOF'
.terraform/
*.tfstate
*.tfstate.*
*.tfvars
EOF                                  # .terraform.lock.hcl se NE ignoriše
```

5 · CLOUD-INIT

Konfiguracija pri prvom pokretanju

cloud-init je standardni alat u slikama oblaka (Ubuntu, Debian, Fedora, većina cloud slika) koji se pri *prvom* pokretanju mašine čita konfiguraciju iz izvora podataka (*datasource*) i primjenjuje je: postavlja hostname, kreira korisnike i SSH ključeve, instalira pakete, upisuje datoteke i pokreće naredbe. Na AWS-u se konfiguracija čita sa metadata servisa, a u lokalnom KVM/libvirt okruženju iz male ISO slike s oznakom `cidata` (datasource *NoCloud*) koju kreira Terraform ili Packer.

DATOTEKA	SADRŽAJ
`user-data`	Šta se radi na mašini: korisnici, paketi, datoteke, naredbe. Počinje redom `#cloud-config`.
`meta-data`	Identitet instance (`instance-id`, `local-hostname`). Kad se `instance-id` promijeni, cloud-init se smatra novom mašinom i ponovo se izvršava.
`network-config`	Opcionalna mrežna konfiguracija.

cloud-init u fazama izvršava module (`users`, `write_files`, `packages`, `runcmd`...). Većina modula je idempotentna, a ono što se izvršava samo jednom po instanci (npr. `runcmd`) cloud-init pamti u `/var/lib/cloud/`.

6 · PRIMJER 2

Vlastiti user-data

Konfiguracija koju će svaka naša VM dobiti pri prvom pokretanju: korisnik `student` s SSH ključem i bez lozinke, nginx i početna stranica.

yaml — Isječak 8 · user-data

```yaml
#cloud-config
hostname: web1
users:
  - name: student
    groups: [sudo]
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    lock_passwd: true                # prijava samo ključem
    ssh_authorized_keys:
      - ssh-ed25519 AAAA...vas_javni_kljuc student@racunar

package_update: true
packages:
  - nginx
  - curl

write_files:
  - path: /var/www/html/index.html
    permissions: "0644"
    content: |
      <h1>web1: konfigurisano cloud-initom</h1>

runcmd:
  - systemctl enable --now nginx
  - echo "cloud-init gotov $(date -Is)" >> /var/log/ruo-init.log
```

Ispravnost se provjerava prije upotrebe. Na računaru s instaliranim cloud-initom radi `cloud-init schema --config-file user-data`, a minimum je `python3 -c "import yaml,sys; yaml.safe_load(open(sys.argv[1]))" user-data`. Na već pokrenutoj mašini stanje daje `cloud-init status --long`, a dnevnik je u `/var/log/cloud-init-output.log`.

> **GREŠKA KOJA SE ČESTO PRAVI**
>
> YAML je osjetljiv na uvlačenje, a prvi red mora biti tačno `#cloud-config`. Pogrešan user-data cloud-init *ne prijavi kao grešku pri pokretanju*: mašina se podigne bez korisnika, pa se ne možemo prijaviti. Dnevnik je prvo mjesto gdje se gleda.

---

7 · PRIMJER 3

Virtualna mašina u libvirt-u iz Terraforma

Spajamo dosad naučeno. Terraform pomoću providera `dmacvicar/libvirt` preuzme baznu (cloud) sliku, napravi diskove, generiše `cidata` ISO s našim user-data i definiše mašinu. U 1. sedmici smo isto radili ručno preko `virsh`; sada ispada u nekoliko deklaracija.

Isječak 8 postaje *šablon* (`cloud-init.tftpl`): vrijednosti koje se mijenjaju (hostname, ključ) zamjenjuju se Terraformovom interpolacijom `${...}`.

bash + hcl — Isječak 9 · cloud-init.tftpl i main.tf

```bash
mkdir -p ~/iac/s2/p3 && cd ~/iac/s2/p3
cat > cloud-init.tftpl <<'EOF'
#cloud-config
```

```
hostname: ${hostname}
users:
  - name: student
    sudo: ALL=(ALL) NOPASSWD:ALL
    shell: /bin/bash
    lock_passwd: true
    ssh_authorized_keys:
      - ${ssh_key}
package_update: true
packages: [nginx, curl]
write_files:
  - path: /var/www/html/index.html
    content: |
      <h1>${hostname}</h1>
runcmd:
  - systemctl enable --now nginx
EOF

cat > main.tf <<'EOF'
terraform {
  required_providers {
    libvirt = {
      source  = "dmacvicar/libvirt"
      version = "~> 0.8.3"
    }
  }
}

provider "libvirt" {
  uri = "qemu:///system"
}

variable "vm_name" {
  type    = string
  default = "web1"
}

# bazna slika se preuzima jednom, a disk VM-a je njen "copy-on-write" potomak
resource "libvirt_volume" "base" {
  name   = "ubuntu-noble-base.qcow2"
  pool   = "default"
  format = "qcow2"
  source = "https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img"
}

resource "libvirt_volume" "disk" {
  name           = "${var.vm_name}.qcow2"
  pool           = "default"
  base_volume_id = libvirt_volume.base.id
  size           = 10 * 1024 * 1024 * 1024
}

resource "libvirt_cloudinit_disk" "init" {
  name = "${var.vm_name}-init.iso"
  pool = "default"
  user_data = templatefile("${path.module}/cloud-init.tftpl", {
    hostname = var.vm_name
    ssh_key  = trimspace(file(pathexpand("~/.ssh/id_ed25519.pub")))
  })
}
```

```
resource "libvirt_domain" "vm" {
  name      = var.vm_name
  memory    = 1024
  vcpu      = 1
  cloudinit = libvirt_cloudinit_disk.init.id

  disk { volume_id = libvirt_volume.disk.id }

  network_interface {
    network_name   = "default"
    wait_for_lease = true
  }

  console {
    type        = "pty"
    target_type = "serial"
    target_port = "0"
  }
}

output "ip" {
  value = libvirt_domain.vm.network_interface[0].addresses[0]
}
EOF

terraform init
terraform apply -auto-approve
```

izlaz (skraćeno, vrijednosti se razlikuju)

```
libvirt_volume.base: Creating...
libvirt_volume.disk: Creating...
libvirt_cloudinit_disk.init: Creating...
libvirt_domain.vm: Creating...
...
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

ip = "192.168.122.57"
```

bash — Isječak 10 · provjera i drift

```
ssh -o StrictHostKeyChecking=accept-new student@$(terraform output -raw ip) 'cloud-init status --long;
hostname'
curl -s http://$(terraform output -raw ip)           # stranica koju je upisao cloud-init

terraform apply -auto-approve                        # nema promjena: idempotentno
virsh -c qemu:///system destroy web1                 # drift: neko ugasi mašinu ručno
virsh -c qemu:///system undefine web1                # ... i obriše definiciju
terraform plan                                       # Plan: 1 to add (domena se vraća)
terraform apply -auto-approve                        # nova VM iz istog koda

terraform destroy -auto-approve
```

Posebno je važno uočiti sljedeće. (1) Disk VM-a je *potomak* bazne slike, pa je kreiranje druge mašine pitanje promjene jedne varijable (`-var vm_name=web2`). (2) Izlaz `ip` je ulaz za sljedeći korak u lancu: u 3. sedmici se iz

njega generiše Ansible inventory. (3) Brisanjem domene smo simulirali drift: Terraform ga je uočio bez ikakve dodatne konfiguracije.

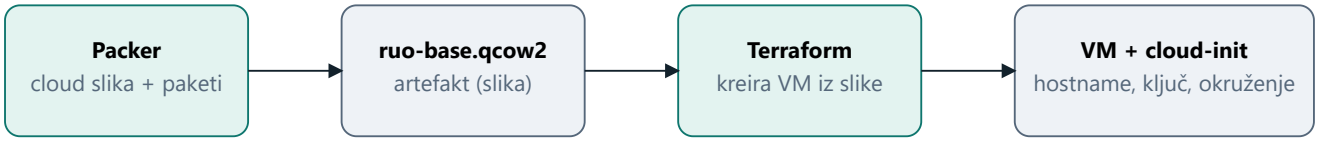
> **ČESTE PREPREKE**
>
> - `Permission denied` pri otvaranju diska: AppArmor/permisije nad `/var/lib/libvirt/images`. Rješenje za lab-okruženje: `security_driver = "none"` u `/etc/libvirt/qemu.conf` i `sudo systemctl restart libvirtd`.
> - Pool `default` ne postoji: `virsh pool-define-as default dir --target /var/lib/libvirt/images && virsh pool-start default`, a ako mreža `default` nije aktivna, `virsh net-start default`.
> - Nema `/dev/kvm`: u domeni dodati `type = "qemu"` (emulacija, sporo, ali radi).
> - Provider je pinovan na verziju 0.8.x jer novija serija ima drugačiju sintaksu resursa.

8 · PACKER

Slika kao artefakt

U primjeru 3 mašina pri svakom pokretanju preuzima i instalira nginx. To je sporo i zavisi od mreže u tom trenutku. **Packer** taj posao premješta u fazu *izgradnje slike*: jednom se pokrene privremena VM, instaliraju se paketi, rezultat se snimi kao slika (*golden image*), a Terraform ubuduće pokreće mašine iz gotove slike. To je nepromjenjiva infrastruktura u praksi.

Packer se konfiguriše u HCL-u kao i Terraform. Pojmovi: **source** (kako se pravi privremena mašina, ovdje QEMU), **build** (šta se radi s njom) i **provisioner** (skripta ili Ansible unutar mašine).



rijetko se mijenja ← u slici | pri svakom pokretanju → u cloud-initu

Slika 2. Podjela posla: Packer pravi sliku (sporo i jednom), Terraform pravi mašine iz nje (brzo i puno puta), a cloud-init ih personalizuje.

9 · PRIMJER 4

Vlastita "zlatna" slika s Packerom

Gradimo sliku `ruo-base.qcow2` iz iste Ubuntu cloud slike, s već instaliranim nginx-om. Packer pokreće QEMU, a pristup privremenoj mašini omogućava ISO `cidata` koji Packer sam sastavi iz datoteka u `cd_files`.

bash + hcl — Isječak 11 · Packer: ubuntu.pkr.hcl

```
mkdir -p ~/iac/s2/p4/seed && cd ~/iac/s2/p4
cat > seed/user-data <<'EOF'
#cloud-config
ssh_pwauth: true
chpasswd:
  expire: false
  users:
    - { name: ubuntu, password: packer, type: text }
EOF
echo "instance-id: packer-build" > seed/meta-data

cat > ubuntu.pkr.hcl <<'EOF'
packer {
  required_plugins {
    qemu = {
      source  = "github.com/hashicorp/qemu"
      version = "~> 1"
    }
  }
}

source "qemu" "ubuntu" {
```

```
  iso_url          = "https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img"
  iso_checksum     = "file:https://cloud-images.ubuntu.com/noble/current/SHA256SUMS"
  disk_image       = true            # ulaz je gotov disk, ne instalacijski ISO
  format           = "qcow2"
  disk_size        = "10G"
  memory           = 1024
  accelerator      = "kvm"           # bez KVM-a: "tcg" (sporo)
  headless         = true
  cd_files         = ["./seed/user-data", "./seed/meta-data"]
  cd_label         = "cidata"
  ssh_username     = "ubuntu"
  ssh_password     = "packer"
  ssh_timeout      = "10m"
  shutdown_command = "sudo shutdown -P now"
  output_directory = "output"
  vm_name          = "ruo-base.qcow2"
}

build {
  sources = ["source.qemu.ubuntu"]

  provisioner "shell" {
    inline = [
      "sudo cloud-init status --wait",
      "sudo apt-get update",
      "sudo DEBIAN_FRONTEND=noninteractive apt-get install -y nginx qemu-guest-agent",
      "sudo systemctl enable nginx",
      "echo 'ruo-base 1.0' | sudo tee /etc/ruo-image-version",
      # čišćenje: build lozinka se zaključava, a cloud-init se resetuje da ponovo radi pri prvom
pokretanju
      "sudo passwd -l ubuntu",
      "sudo cloud-init clean --logs --seed",
    ]
  }
}
EOF

packer init .
packer validate .
packer build .
ls -lh output/
```

izlaz (skraćeno, vrijednosti se razlikuju)

```
qemu.ubuntu: output will be in this color.
==> qemu.ubuntu: Retrieving ISO
==> qemu.ubuntu: Starting VM, booting from CD-ROM
==> qemu.ubuntu: Connected to SSH!
==> qemu.ubuntu: Provisioning with shell script: ...
==> qemu.ubuntu: Gracefully halting virtual machine...
Build 'qemu.ubuntu' finished after 3 minutes 41 seconds.
-rw-r--r-- 1 student student 640M ... ruo-base.qcow2
```

Zadnja dva reda provisionera su presudna za cijeli koncept. `passwd -l` zaključava lozinku korištenu samo pri izgradnji, a `cloud-init clean` briše tragove prvog pokretanja da bi se cloud-init *ponovo izvršio* pri pokretanju mašine iz slike i primijenio nove korisnike i ključeve. Bez toga bi mašine iz slike bile nepristupačne. Provjeriti i da `grep -r PasswordAuthentication /etc/ssh/sshd_config.d/` ne ostavlja prijavu lozinkom uključenom u slici koja ide dalje.

Korištenje slike iz Terraforma

Dovoljna je jedna izmjena u primjeru 3: bazni volumen više ne dolazi s interneta, nego iz naše slike, a nginx se više ne instalira pri prvom pokretanju.

bash + hcl — Isječak 12 · izmjena na primjeru 3

```
mkdir -p ~/iac/s2/p4/vm && cd ~/iac/s2/p4/vm
cp ../../p3/main.tf ../../p3/cloud-init.tftpl .       # samo kod, bez statea
# 1) bazni volumen iz lokalne slike umjesto URL-a:
sed -i 's|source = "https://.*"|source = "${abspath("${path.module}/../output/ruo-base.qcow2")}"|'
main.tf
# 2) iz cloud-init.tftpl ukloniti "package_update" i "packages" (nginx je već u slici)
terraform init && terraform apply -auto-approve
curl -s http://$(terraform output -raw ip)
ssh student@$(terraform output -raw ip) cat /etc/ruo-image-version     # ruo-base 1.0
terraform destroy -auto-approve
```

Mašina je sada spremna za nekoliko sekundi, a ne minuta, i svaka mašina iz slike je identična. Kad se zahtjev promijeni (npr. nova verzija nginx-a), u Packeru se mijenja jedna linija, pravi se slika `1.1`, a Terraform zamijeni mašine. Mašine se ne "krpe".

10 · PREGLED

Ko šta radi u lancu

ALAT	FAZA	ODGOVARA NA PITANJE	GDJE SE VIDI
Packer	izgradnja slike	Šta je u slici (OS, paketi, hardening)?	Primjer 4
Terraform / OpenTofu	kreiranje resursa	Koje mašine, diskovi i mreže postoje i u kojoj mjeri?	Primjeri 1, 3
cloud-init	prvo pokretanje	Ko je ova mašina (hostname, korisnici, ključevi)?	Primjeri 2, 3
Ansible	konfiguracija u radu	Kako je mašina podešena i kako se to održava?	Sedmica 3

Rad u Git repozitoriju

- U repozitorij idu `*.tf`, `*.pkr.hcl`, `cloud-init.tftpl` i `.terraform.lock.hcl`. Ne idu `.terraform/`, `*.tfstate`, slike (`*.qcow2`) i `*.tfvars` s tajnama.
- Svaka izmjena infrastrukture je commit, a prije spajanja se pregleda izlaz `plan`.
- Primjeri se čuvaju u strukturi koja ide u projektni zadatak (25 bodova): kod, dokumentacija arhitekture i mogućnost da ga drugi student pokrene s `terraform apply`.

Pitanja za provjeru

PITANJE	ODGOVOR
Zašto se `terraform.tfstate` ne stavlja u Git?	Sadrži tajne u čistom tekstu, a pri radu u timu uzrokuje sukobe. Čuva se u zaključanom udaljenom backendu.
Šta pokazuje `terraform plan` kad je neko ručno promijenio resurs?	Drift: razliku između statea nakon osvježavanja i koda. `apply` stvarno stanje vraća na ono iz koda.
Zašto je `cloud-init clean` u Packer skripti?	Da se cloud-init izvrši ponovo pri prvom pokretanju svake mašine iz slike.
Kada slika, a kada cloud-init?	Slika: ono što je isto za sve mašine i rijetko se mijenja. cloud-init: ono što je specifično za instancu.

Zadaci za samostalni rad

1. U primjeru 1 dodati drugu datoteku pomoću `for_each` nad listom imena i pratiti kako `plan` reaguje kad se element ukloni iz liste.
2. U primjeru 3 napraviti dvije mašine (`web1`, `web2`) pomoću `for_each` ili `count` i dodati izlaz s obje IP adrese.
3. Pokrenuti iste primjere s `tofu` umjesto `terraform` i zapisati razlike ako ih ima.
4. Proširiti Packer sliku s još jednim paketom i verzijom `1.1`, pa Terraformom zamijeniti mašinu.

> **SLJEDEĆA SEDMICA**
> Mašine su sada kreirane iz koda, ali se još ručno "dorađuju". Sedmica 3 uvodi Ansible (konfiguracija u radu), povezivanje s Terraformom i kontejnere kao kod (Dockerfile, Compose, Kubernetes manifesti).